

DATABASE DESIGN DOCUMENT

Motorcycle Shop E-Commerce System

MongoDB Collections and Schema Reference

Derived from FR1.1 to FR1.10 | MERN Stack (MongoDB, Express.js, React, Node.js)

PREPARED BY

Benitez, Alwin C.
Roduta, Kelly Laurence Q.

SECTION

BSIT – S – 3A

SUBMITTED TO

Mr. Dalisay

DATE

September 2026

Contents

1. Design Principles	3
2. Collections Overview	3
3. Relationships	4
4. Collection Specifications	5
4.1 users	5
4.2 categories	5
4.3 brands	5
4.4 products	6
4.5 carts	6
4.6 orders	6
4.7 installment_plans	8
4.8 payment_schedules	8
4.9 payments	8
4.10 reviews	9
4.11 suppliers	9
4.12 restocks	10
4.13 inventory_transactions	10
4.14 notifications	10
4.15 settings	11
5. Aggregation Map	12
6. Normalization Notes	13
7. Implementation Notes	13

1. Design Principles

This document describes the MongoDB collections needed to support the Motorcycle Shop E-Commerce System. It is a reference to follow while building the Mongoose models and API, written from the functional requirements (FR1.1 to FR1.10).

1. **No computed fields are stored.** Line totals, order totals, tax, average ratings, remaining balances, financed amount, monthly payment and total expenses are derived with aggregation (`$multiply`, `$sum`, `$avg`, `$lookup`). Section 5 maps each one.
2. **Snapshots are not computed fields.** Values that must stay frozen after an order is placed (unit price, product name, installment plan terms) are copied into the order. A later price or plan edit must not change old orders.
3. **Embed or reference deliberately.** Data that is always read with its parent and has no life of its own (order items, cart items, addresses, images) is embedded. Anything shared, unbounded or queried on its own (payments, schedule rows, reviews) is a separate collection linked by `objectId`.
4. **Timestamps everywhere.** Every collection uses `timestamps: true`, which adds `createdAt` and `updatedAt`. These are not repeated in the tables below. `_id` is implicit.
5. **Archive instead of delete** for records that other documents reference (products, plans, suppliers, users), using `isArchived` OR `isActive` flags.

2. Collections Overview

#	Collection	Purpose	Requirements
1	users	Accounts, roles, profile, ID documents, password reset	FR1.1, FR1.8
2	categories	Product categories	FR1.3
3	brands	Motorcycle and parts brands	FR1.3
4	products	Motorcycles, parts, accessories, riding gear	FR1.2
5	carts	One persistent cart per user	FR1.4
6	orders	Orders, shipping, status, installment application, cancellation	FR1.5, FR1.10.3
7	installment_plans	Admin-configured plans (term, rate, downpayment, fee)	FR1.10.1
8	payment_schedules	One row per installment due date	FR1.10.4, FR1.10.6
9	payments	Every payment, downpayment, payoff and manual entry; receipts	FR1.5.5, FR1.10.5, FR1.10.8
10	reviews	Ratings and reviews with admin moderation	FR1.7
11	suppliers	Supplier details and product lines	FR1.6.2
12	restocks	Restocking records and unit costs	FR1.6.1, FR1.6.3
13	inventory_transactions	Append-only stock ledger	FR1.6.4, FR1.6.5
14	notifications	In-app alerts and Mailtrap email log	FR1.5.10, FR1.6.6
15	settings	Tax, low-stock threshold, penalty and default rules	FR1.10.7, FR1.10.9

3. Relationships

From	Cardinality	To	Linked by
users	1 to 0..1	carts	carts.user
users	1 to many	orders	orders.user
users	1 to many	reviews	reviews.user
users	1 to many	notifications	notifications.recipient
categories	1 to many	products	products.category
brands	1 to many	products	products.brand
products	1 to many	reviews	reviews.product
products	1 to many	inventory_transactions	inventory_transactions.product
products	many to many	installment_plans	products.eligiblePlans
installment_plans	1 to many	orders	orders.installment.plan
orders	1 to many	payment_schedules	payment_schedules.order
orders	1 to many	payments	payments.order
payment_schedules	many to many	payments	payments.allocations.schedule
orders	1 to many	inventory_transactions	inventory_transactions.order
suppliers	1 to many	restocks	restocks.supplier
restocks	1 to many	inventory_transactions	inventory_transactions.restock

4. Collection Specifications

Types follow Mongoose conventions. Fields marked **required** or **unique** map directly to schema options. Bracketed types such as [string] are arrays.

4.1 users

Covers: FR1.1.1 to FR1.1.7, FR1.8.1. Stores both Admin and Customer accounts.

Field	Type	Notes
name	String, required	FR1.1.1
email	String, required, unique, lowercase	FR1.1.1
password	String, required, select: false	Store the bcrypt hash only
role	String enum ['admin', 'customer'], default 'customer'	FR1.1.7, FR1.8.1
contactNumber	String	FR1.1.3
address	Embedded { street, city, postalCode, country }	FR1.1.1
profilePicture	{ url, publicId }	FR1.1.3
validId	{ url, publicId }	FR1.1.3. Used for installment applications
proofOfIncome	{ url, publicId }	FR1.1.3. Used for installment applications
isActive	Boolean, default true	FR1.1.5. Account deactivation
resetPasswordToken	String (hashed), select: false	FR1.1.4. Link sent by email
resetPasswordExpire	Date, select: false	FR1.1.4

Indexes: { email: 1 } unique

4.2 categories

Covers: FR1.3.1, FR1.3.2, FR1.3.5. Examples: Motorcycles, Engine Parts, Brakes, Tires and Wheels, Helmets, Riding Gear, Accessories, Lubricants.

Field	Type	Notes
name	String, required, unique	
description	String	

4.3 brands

Covers: FR1.3.3, FR1.3.4. Examples: Honda, Yamaha, Suzuki, Kawasaki, CFMOTO.

Field	Type	Notes
name	String, required, unique	
logo	{ url, publicId }	Optional

4.4 products

Covers: FR1.2.1 to FR1.2.10, FR1.6.6. Catalog items. Average rating and review count are not stored; they come from reviews.

Field	Type	Notes
name	String, required	
description	String	
category	ObjectId (categories), required	FR1.3.2
brand	ObjectId (brands), required	
compatibleModels	[String]	e.g. "Honda Click 125". Used by FR1.2.5 search
price	Number, required, min 0	
stock	Number, required, min 0, default 0	Maintained state, updated with every inventory transaction (FR1.6.4)
lowStockThreshold	Number	Falls back to the default in settings when absent
images	[{ url, publicId }]	FR1.2.6. Multiple images
installmentEligible	Boolean, default false	FR1.2.3
minInstallmentPrice	Number	Minimum price to qualify for installment (FR1.2.3)
eligiblePlans	[ObjectId (installment_plans)]	Optional (FR1.2.1)
isArchived	Boolean, default false	FR1.2.4

Indexes: text index on name and description; compound { category, brand, price }; { compatibleModels }

4.5 carts

Covers: FR1.4.1 to FR1.4.5. One cart per user, persisted across sessions. No prices are stored; the current price is read with \$lookup.

Field	Type	Notes
user	ObjectId (users), required, unique	FR1.4.5
items	[{ product: ObjectId (products), quantity: Number, min 1 }]	FR1.4.1 to FR1.4.3

4.6 orders

Covers: FR1.5.1 to FR1.5.11, FR1.10.3. Order date comes from createdAt. No subtotal or total fields are stored.

Field	Type	Notes
user	ObjectId (users), required	
items	[{ product, name, unitPrice, quantity }]	Snapshot at checkout (FR1.5.4)
shippingInfo	{ address, city, phoneNumber, postalCode, country }, all required	FR1.5.2
shippingPrice	Number, default 0	Input value, not computed
taxRate	Number	Snapshot of the rate applied at checkout
paymentMethod	enum ['cod', 'online', 'installment']	FR1.5.5

Field	Type	Notes
status	enum ['For Approval', 'Awaiting Downpayment', 'Pending', 'Processing', 'Shipped', 'Delivered', 'Cancelled']	FR1.5.6. The first two apply to installment orders only
statusHistory	[[status, changedAt, changedBy: ObjectId (users)]]	Optional audit trail
deliveredAt	Date	Set when the status becomes Delivered
installment	Embedded subdocument (below)	Present only if paymentMethod is installment
cancellation	Embedded subdocument (below)	Present only if the order was cancelled

installment subdocument

Field	Type	Notes
plan	ObjectId (installment_plans)	FR1.5.11
planSnapshot	{ name, termMonths, interestRate, minDownpaymentPercent, processingFee }	Frozen copy so later plan edits do not change this contract
downpayment	Number	Customer-chosen amount (FR1.5.11)
downpaymentConfirmedAt	Date	Set by Admin. Required before Processing (FR1.5.6)
application	{ status: enum ['For Approval', 'Approved', 'Rejected'], reviewedBy: ObjectId (users), reviewedAt: Date, rejectionReason: String }	FR1.10.3
validId	{ url, publicId }	Documents submitted with this application
proofOfIncome	{ url, publicId }	
defaultedAt	Date	FR1.10.9. Absent means not defaulted
defaultedBy	ObjectId (users)	Admin who marked the default

cancellation subdocument

Field	Type	Notes
cancelledAt	Date	
reason	String	
downpaymentOutcome	enum ['none', 'refunded', 'forfeited']	FR1.5.9
refundedAmount	Number	Recorded according to store policy

Indexes: { user, createdAt: -1 }, { status }, { items.product }, { installment.application.status }

4.7 installment_plans

Covers: FR1.10.1. Archive plans instead of deleting them so old orders keep a valid reference.

Field	Type	Notes
name	String, required	e.g. "12 Months Standard"
termMonths	Number, required	3, 6, 12, 24 or 36
interestRate	Number, required	Decide in code whether this is flat or annual and apply it consistently
minDownpaymentPercent	Number, required	
processingFee	Number, default 0	Optional
isArchived	Boolean, default false	

4.8 payment_schedules

Covers: FR1.10.4, FR1.10.6, FR1.10.7. One document per due date, created when an installment order is approved.

Field	Type	Notes
order	ObjectId (orders), required	
user	ObjectId (users), required	Convenience for customer queries and reminder emails
installmentNo	Number, required	1, 2, 3, and so on
dueDate	Date, required	
amountDue	Number, required	Fixed contractual amount at approval time, so the agreed schedule never shifts if plans or settings change
penalty	{ amount: Number, appliedAt: Date }	Added only when late fees are enabled and the grace period has passed
reminderSentAt	Date	For upcoming and missed due-date emails

Indexes: { order, installmentNo } unique; { dueDate }

Note Paid, Unpaid and Overdue are not stored. A row is Paid when the payments allocated to it cover amountDue, and Overdue when dueDate has passed and it is not Paid. The same applies to the order-level Unpaid, Partially Paid and Paid. Only **Defaulted** is stored (orders.installment.defaultedAt) because it is an admin decision.

4.9 payments

Covers: FR1.5.5, FR1.10.5, FR1.10.8, FR1.8.5. All payments: cash on delivery, online, downpayments, scheduled installments, early payoffs and manual admin entries.

Field	Type	Notes
order	ObjectId (orders), required	
user	ObjectId (users), required	The paying customer
type	enum ['full', 'downpayment', 'installment', 'payoff']	
amount	Number, required	Amount actually paid
method	enum ['cod', 'online', 'cash', 'bank_deposit']	

Field	Type	Notes
referenceNo	String	Gateway or bank reference. Never store card details
paidAt	Date, required	
allocations	[[schedule: ObjectId (payment_schedules), amount: Number]]	Which schedule rows this payment settled. Several entries cover paying ahead or early payoff
recordedBy	ObjectId (users)	Set only for manual entries recorded by Admin
receiptNo	String, unique	Generated identifier for the receipt (viewable and emailable)

Indexes: { order, paidAt }; { receiptNo } unique; { allocations.schedule }

Note Remaining balance is derived: total cost (from the order and plan snapshot) minus the \$sum of payments.

4.10 reviews

Covers: FR1.7.1 to FR1.7.6. One review per customer per product; reviewing again edits the existing document.

Field	Type	Notes
product	ObjectId (products), required	
user	ObjectId (users), required	
rating	Number, required, min 1, max 5	FR1.7.1
title	String, required	FR1.7.2
body	String, required	FR1.7.2
isHidden	Boolean, default false	Admin moderation (FR1.7.5)

Indexes: { product, user } **unique**, which enforces FR1.7.6

Note The Verified Purchase badge is not stored. At read time, \$lookup into orders and check for a Delivered order from this user containing the product (FR1.7.4).

4.11 suppliers

Covers: FR1.6.2. Suppliers the admin buys stock from.

Field	Type	Notes
name	String, required	
contact	{ person, email, phone, address }	
productLines	[String]	Supplied product lines
isActive	Boolean, default true	

4.12 restocks

Covers: FR1.6.1, FR1.6.3. No line cost or total cost is stored; total expense is aggregated.

Field	Type	Notes
supplier	ObjectId (suppliers), required	
items	{ product: ObjectId (products), quantity: Number, unitCost: Number }	
restockedAt	Date, required	Used for expense report date ranges
referenceNo	String	Supplier invoice or purchase order number
recordedBy	ObjectId (users), required	Admin

Indexes: { restockedAt }; { supplier }

4.13 inventory_transactions

Covers: FR1.6.4, FR1.6.5. Append-only ledger. Documents are never edited.

Field	Type	Notes
product	ObjectId (products), required	
type	enum ['restock', 'sale', 'reserve', 'release', 'cancellation', 'adjustment']	reserve is used when an installment is approved (FR1.6.4)
quantityChange	Number, required	Signed: positive adds stock, negative removes it
order	ObjectId (orders)	When caused by an order
restock	ObjectId (restocks)	When caused by a restock
performedBy	ObjectId (users)	
note	String	

Indexes: { product, createdAt: -1 }

Note There are no stock-before or stock-after columns because those are derived. `products.stock` is updated in the same operation, using a transaction or an `$inc` guarded by `stock >= quantity` to prevent overselling.

4.14 notifications

Covers: FR1.5.10, FR1.6.6. In-app admin alerts (such as low stock) and a log of emails sent through Mailtrap.

Field	Type	Notes
recipient	ObjectId (users), required	
channel	enum ['in_app', 'email']	
type	enum ['order_summary', 'order_status', 'installment_approved', 'installment_rejected', 'downpayment_confirmed', 'due_reminder', 'missed_payment', 'receipt', 'low_stock']	

Field	Type	Notes
subject	String	
message	String	
order	ObjectId (orders)	Optional
product	ObjectId (products)	Used for low-stock alerts
isRead	Boolean, default false	

4.15 settings

Covers: FR1.10.7, FR1.10.9, FR1.6.6, FR1.5.9. A single document holding admin-configurable values.

Field	Type	Notes
taxRate	Number	Default rate copied into orders . taxRate at checkout
defaultLowStockThresh hold	Number	FR1.6.6
latePenalty	{ enabled: Boolean, type: enum ['fixed', 'percentage'], value: Number, gracePeriodDays: Number }	FR1.10.7
defaultAfterMissedPa yments	Number	FR1.10.9
cancellationPolicy	{ downpaymentRefundable: Boolean }	Optional (FR1.5.9)

5. Aggregation Map

Every value below is calculated at read time. None of them is a stored field.

Derived value	How it is derived
Cart line total and cart total	<code>\$lookup</code> product, <code>\$multiply</code> price by quantity, <code>\$sum</code> ; add tax from settings.
Order items price	<code>\$sum</code> of <code>\$multiply</code> : [<code>"\$items.unitPrice"</code> , <code>"\$items.quantity"</code>] using <code>\$unwind</code> or <code>\$reduce</code> .
Order total	<code>Items price + shippingPrice + (items price x taxRate)</code> .
Financed amount	<code>Order total minus installment.downpayment</code> .
Interest, fees, monthly amount, total installment cost	Computed from the financed amount and <code>planSnapshot</code> in code or <code>\$project</code> (FR1.10.2). The stored <code>amountDue</code> on each schedule row is the frozen result.
Remaining balance	Total cost minus the <code>\$sum</code> of <code>payments.amount</code> for the order.
Schedule status (Paid, Unpaid, Overdue)	<code>\$lookup</code> payments by <code>allocations.schedule</code> , compare the sum to <code>amountDue</code> , and compare <code>dueDate</code> with <code>\$\$NOW</code> .
Order installment status	Sum of payments against total cost gives Unpaid, Partially Paid or Paid. Defaulted if <code>installment.defaultedAt</code> exists.
Average rating and review count	<code>\$group</code> on reviews by product with <code>\$avg</code> and <code>\$sum</code> , excluding <code>isHidden: true</code> .
Verified purchase	<code>\$lookup</code> orders where status is Delivered and <code>items.product</code> matches.
Total restock expense	<code>\$sum</code> of <code>\$multiply</code> : [<code>"\$items.quantity"</code> , <code>"\$items.unitCost"</code>] from restocks, grouped by product or date range.
Low stock list	<code>\$match</code> with <code>\$expr</code> comparing stock to <code>\$ifNull(lowStockThreshold, defaultThreshold)</code> .
Sales report	<code>\$match</code> on a <code>createdAt</code> range and valid statuses, <code>\$group</code> by day or month.
Top products by category	<code>\$unwind</code> items, <code>\$group</code> by product, <code>\$lookup</code> category, <code>\$sort</code> by quantity.
Installment collections, receivables, overdue	Group payments by date; <code>\$lookup</code> schedules to payments for balances; overdue where <code>dueDate</code> is before <code>\$\$NOW</code> and unpaid.
Admin dashboard charts	A combination of the aggregations above.

6. Normalization Notes

MongoDB designs trade some normalization for read performance. Where the design departs from third normal form, it is on purpose.

Level	Status	Comment
First normal form	Strictly no	Embedded arrays and subdocuments (items, images, allocations, address) are repeating groups. Treated as child tables, they are in 1NF.
Second normal form	Yes	Single <code>_id</code> keys avoid partial dependencies. Composite keys (order + installmentNo, product + user) have attributes that depend on the whole key. The order item name depends only on product, kept as a snapshot.
Third normal form	Mostly	Intentional redundancy: user on schedules and payments (query convenience), planSnapshot and ID documents on the order (historical record), products.stock (fast reads and overselling guard).

Strict 3NF For strict 3NF, remove user from payment_schedules and payments, drop the item name, and drop products.stock in favor of summing inventory_transactions. The snapshots are the only departures that should be kept, since they record what was true at the time of sale.

7. Implementation Notes

- Use Mongoose `ref` with `populate()` for simple reads and `$lookup` for reports.
- Wrap order placement, installment approval, payment recording and cancellation in a **MongoDB session transaction**. Each one touches several collections. Transactions need a replica set, which MongoDB Atlas provides by default.
- Hash passwords with `bcrypt` and use JWT for token-based authentication (FR1.8.3). Do not persist any card data (FR1.8.5).
- Restrict installment approval and manual payment recording to the Admin role with middleware (FR1.8.2).
- Prefer archive flags (`isArchived`, `isActive`) over hard deletes for records already referenced elsewhere.